

10.3.6 语音交互、大模型与项目验收

第六次课讲义 · 代码均摘自本仓库 `src/mecamind_tools`，可对照源文件逐行验证

建议时长 2~2.5 小时：原理与架构 30~40 分钟 · 跟做 70~90 分钟 · 验收与作品集 20~30 分钟

0. 本节课要带走什么

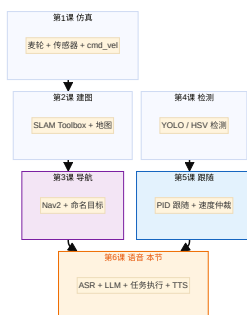
前五课分别打通了仿真底盘、建图、导航、检测、视觉跟随。本节把它们接到**语音交互层**：

1. 听懂你说的话（麦克风 / 流式 ASR / 文本兜底）
2. 理解你要做什么（规则解析或阿里云 LLM，失败自动降级）
3. **真正让车动起来**（`mission_executor` 按 intent 调 Nav2、开跟随、方向点动、急停）
4. 用嘴回话（TTS + 本机播放），并完成项目级验收

一句话主线：

语音文本 → TaskCommand JSON → MissionExecutor 分发动作 → 底盘 / Nav2 / 跟随

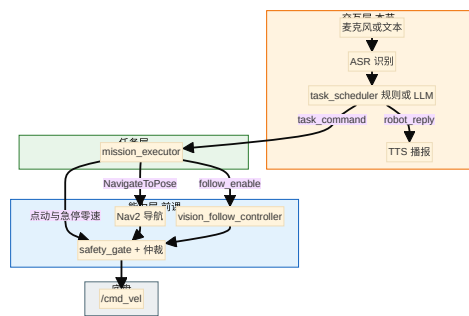
1. 六课总图与本节落点



本节在系统里的角色是**交互与任务编排**：不替代 Nav2 / PID，只负责把自然语言变成它们能消费的结构化命令。

2. 核心架构：语音如何变成动作

2.1 五层视角



2.2 必须背下来的三段式

阶段	话题	载荷	谁生产	谁消费
听懂	/mecamind/voice_command	纯文本，如 去卧室	voice_listen / ASR / 人工 pub	task_scheduler
理解	/mecamind/task_command	JSON: intent / target / requires_confirmation / reply	task_scheduler	mission_executor
执行	Nav2 Action / follow_enable / /cmd_vel_teleop 点动 / 零 Twist	目标位姿、开关或速度	mission_executor	Nav2 / 跟随 / 速度链路
回话	/mecamind/robot_reply → TTS	口语字符串	task_scheduler 与 mission_executor	aliyun_tts → 播放

intent 白名单共 9 个：stop / cancel / confirm / patrol / follow / mapping / navigate / move / unknown。其中 move 是**方向点动**（前进 / 后退 / 左转 / 右转），本讲义 3.4 节详解。

LLM 不直接开车。 大模型只负责把口语变成 JSON；真正动轮子的永远是 mission_executor。

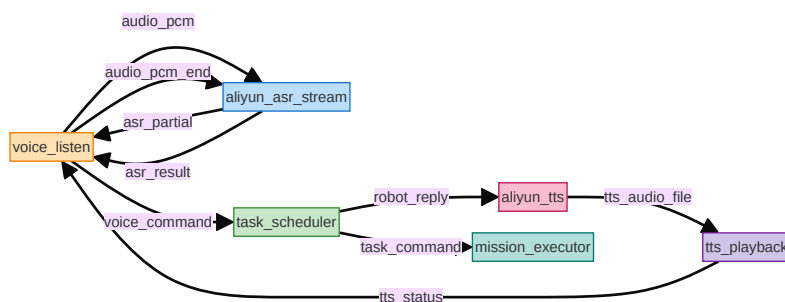
2.3 一句话对照表：真实输入与真实输出

以下 JSON 都是 parse_task_command (task_scheduler.py) 对真实输入的实际输出，可以用 LAB-2 的文本路径逐条复现：

你说	/mecamind/task_command 实际载荷
去卧室	{"intent": "navigate", "target": "卧室", "requires_confirmation": false, "reply": "好的, 正在前往卧室."}
前进	{"intent": "move", "target": "forward", "requires_confirmation": false, "reply": "好的, 前进."}
左转	{"intent": "move", "target": "left", "requires_confirmation": false, "reply": "好的, 左转."}
开始跟随	{"intent": "follow", "target": "", "requires_confirmation": false, "reply": "好的, 开始跟随."}
停止	{"intent": "stop", "target": "", "requires_confirmation": false, "reply": "好的, 正在停止."}
确认	{"intent": "confirm", "target": "", "requires_confirmation": false, "reply": "好的, 开始执行."}
随便做点什么	{"intent": "unknown", "target": "随便做点什么", "requires_confirmation": true, "reply": "请确认是否执行: 随便做点什么? 说确认或取消."}

2.4 默认连续听节点图

启动入口：ros2 launch mecamind_tools mecamind_aliyun_voice.launch.py



源码锚点：

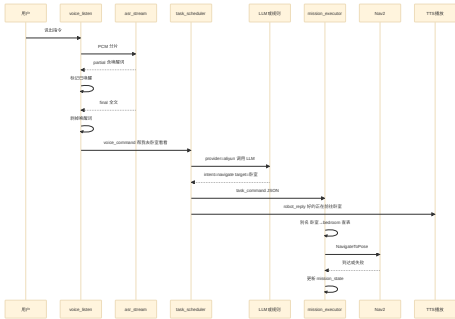
- Launch: src/mecamind_tools/launch/mecamind_aliyun_voice.launch.py (llm_provider 默认 aliyun)
- 听麦: voice_listen_node.py → VoiceListenNode
- 流式 ASR: aliyun_speech_nodes.py → AliyunAsrStreamNode
- 客户端: aliyun_clients.py → AliyunStreamingRecognition / AliyunLlmClient
- 意图: task_scheduler.py → TaskSchedulerNode / parse_task_command
- 执行: mission_executor.py → MissionExecutorNode
- 综合接导航+跟随: mecamind_interaction.launch.py (含 scheduler + executor)

注意：纯语音 launch 默认不起 mission_executor。要让车动起来，还需导航/交互栈（见第 7 节 LAB），或自行 ros2 run mecamind_tools mecamind_mission_executor ...。课堂演示常用：先起导航，再起语音；或文本直接灌 voice_command 验证解析。

3. 端到端主路径：语音 → LLM → 执行动作

下面以一句真实指令 「小度小度，帮我去卧室看看」 走完全程。这是本节最重要的一张图。

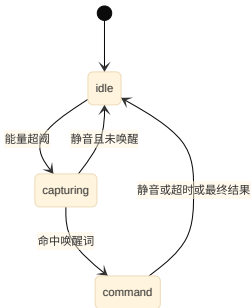
3.1 总时序



3.2 第一步：从声波到 voice_command

文件： voice_listen_node.py

状态机：



范例 1：能量门限自动标定。 WSL/RDP 转发的麦克风底噪电平因机器而异，固定门限要么误触发要么听不见。启动头 auto_calibrate_sec（默认 3 秒）只采底噪，取中位数乘以 margin：

```
# voice_listen_node.py
def calibrated_energy_threshold(samples, floor, margin=2.5):
    """由启动时的环境底噪样本推算能量门限。"""
    valid = sorted(float(s) for s in samples if s >= 0.0)
    if not valid:
        return floor
    median = valid[len(valid) // 2]
    return max(floor, median * margin)
```

启动日志会打出实际标定值，例如 Energy threshold calibrated to 612 (ambient median x 2.5)。

范例 2：唤醒词剥离。 唤醒词默认 小度小度,小度,小杜,小渡,小肚——后几个是 ASR 同音误转写兜底。剥词用「循环找到就删」，所以「小度小度去卧室」不会残留：

```
# voice_listen_node.py
def strip_wake_words(text, wake_words):
    """去掉识别文本里的唤醒词，留下真正指令。"""
    result = text.strip()
    for word in wake_words:
        token = str(word).strip()
        if not token:
            continue
        # 反复剥，避免「小度小度去卧室」残留。
        while True:
            lower = result.lower()
            idx = lower.find(token.lower())
            if idx < 0:
                break
            result = (result[:idx] + result[idx + len(token):]).strip(" , . ! ? ")
    return result.strip()
```

范例 3：为什么唤醒时不插播「我在」。 `_partial_cb` 命中唤醒词时只改标志、不播 TTS——用户通常一口气说完「小度小度停止」，此刻插播会暂停采麦、掐断后半句。只有最终结果里只剩唤醒词时，`_result_cb` 才提示「我在，请说具体指令。」：

```
# voice_listen_node.py -> _partial_cb 片段
if not self._woken and (
    not self._wake_words or text_contains_wake_word(text, self._wake_words)
):
    self._woken = True
    # 注意：这里不播报「我在」——用户通常一口气说完
    # 「小度小度停止」，此刻插播 TTS 会暂停采麦、掐断后半句。
    self.get_logger().info(f"Wake word heard: {text!r}")
```

无麦克风时跳过听麦，直接：

```
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '去卧室'}"
```

效果与 ASR 输出等价——**调度器以下链路完全一样**。这是课堂最稳的调试入口。

3.3 第二步：从文本到 TaskCommand（规则或 LLM）

文件： `task_scheduler.py`、`aliyun_clients.py`

统一数据结构（真实定义）

```
# task_scheduler.py
@dataclass(frozen=True)
class TaskCommand:
    intent: str                # 9 个白名单之一
    target: str = ""          # 导航目标名或点动方向
```

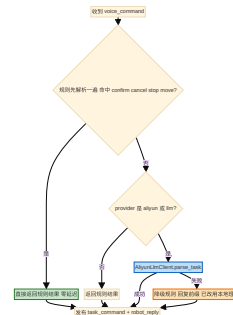
```

requires_confirmation: bool = False # True 表示先问再做
reply: str = "" # 给 TTS 播的话

def to_json(self) -> str:
    # ensure_ascii=False 让中文目标名以原文输出，方便 topic echo 肉眼调试
    return json.dumps(asdict(self), ensure_ascii=False)

```

解析决策流



真实降级代码（这是本文件最值得学的设计）：

```

# task_scheduler.py -> TaskSchedulerNode._parse_command
def _parse_command(self, text: str) -> TaskCommand:
    # 确认/取消/停止/方向点动永远走规则：这些指令要求低延迟、零歧义，
    # 交给 LLM 反而会引入 1 秒级延迟且可能被解析成"导航去 backward"。
    quick = parse_task_command(text)
    if quick.intent in {"confirm", "cancel", "stop", "move"}:
        return quick

    provider = str(self.get_parameter("provider").value).strip().lower()
    if provider not in ("aliyun", "llm"):
        return quick
    try:
        return task_command_from_aliyun_plan(self.llm_client.parse_task(text))
    except Exception as exc:
        self.get_logger().warn(f"Aliyun LLM task parsing failed, falling back to rules: {exc}")
        fallback = parse_task_command(text)
        if fallback.reply and "本地" not in fallback.reply:
            fallback = TaskCommand(
                intent=fallback.intent,
                target=fallback.target,
                requires_confirmation=fallback.requires_confirmation,
                reply=f"已改用本地理解。{fallback.reply}",
            )
        return fallback

```

三个细节值得课堂展开：

1. 快指令白名单含 `move`：「停止」「左转」这类词永远不等网络；

2. **provider 参数每次回调重读**，运行中 `ros2 param set /mecamind_task_scheduler provider rule` 立刻生效，无需重启；
3. **降级只记 warn**——网络抖动是预期内情况，不能让整条语音链路瘫痪；回复加前缀「已改用本地理解」让用户知情。

规则解析（真实优先级顺序）

`parse_task_command` 的匹配顺序即优先级，安全相关的放最前：

```
# task_scheduler.py -> parse_task_command 主干（略去 docstring）
normalized = text.strip().lower()
if any(word in normalized for word in ("stop", "停", "急停", "停车")):
    return TaskCommand("stop", reply="好的，正在停止。")
if any(word in normalized for word in ("cancel", "取消")) or ...:
    return TaskCommand("cancel", reply="好的，已取消。")
direction = parse_move_direction(normalized)          # 前进/后退/左转/右转
if direction:
    return TaskCommand("move", target=direction, reply=MOVE_REPLIES[direction])
# 短确认语：仅在整句 <=6 字时命中，避免「确认去卧室」误判为纯确认
if len(normalized) <= 6 and any(word in normalized for word in CONFIRM_WORDS):
    return TaskCommand("confirm", reply="好的，开始执行。")
if any(word in normalized for word in ("patrol", "巡逻")):
    return TaskCommand("patrol", reply="好的，开始巡逻。")
if any(word in normalized for word in ("follow", "跟随")):
    return TaskCommand("follow", reply="好的，开始跟随。")
if any(word in normalized for word in ("map", "建图")):
    return TaskCommand("mapping", reply="好的，开始建图。")
match = re.search(r"(?:go to|navigate to|去|导航到)\s*([\w\u4e00-\u9fff_-]+)", normalized)
if match:
    target = match.group(1)
    return TaskCommand("navigate", target=target, reply=f"好的，正在前往{target}。")
return TaskCommand("unknown", target=raw, requires_confirmation=True,
    reply=f"请确认是否执行：{raw}？说确认或取消。")
```

方向词表也在同一文件，先查双字词再查介词短语，避免误匹配：

```
# task_scheduler.py
MOVE_DIRECTION_WORDS = {
    "forward": ("前进", "向前", "往前", "直走", "forward", "go straight"),
    "backward": ("后退", "退后", "向后", "往后", "倒退", "倒车", "backward", "go back"),
    "left": ("左转", "向左", "往左", "turn left"),
    "right": ("右转", "向右", "往右", "turn right"),
}
```

任何规则都没命中时返回 `unknown` 且 `requires_confirmation=true`，把原话放进 `target` 留给人工确认——**听不懂就问，比猜着执行安全。**

LLM 解析（真实提示词与防御性校验）

AliyunLlmClient.parse_task 的本质是一次普通 HTTPS POST（只用标准库 urllib，不依赖 openai/requests）。发给模型的 system 提示词原文如下：

```
# aliyun_clients.py -> build_task_parser_messages
system = (
    "You are the task parser for a ROS 2 indoor mobile robot. "
    "Convert the user's command into strict JSON only. "
    "Allowed intents are stop, cancel, confirm, patrol, follow, mapping, navigate, move, unknown. "
    "Use target for room or waypoint names. "
    "For short relative motions (go forward, back up, turn left, turn right) use intent move "
    "with target one of forward, backward, left, right. "
    "Set requires_confirmation true when the command is ambiguous or risky. "
    "The JSON schema is: "
    '{"intent": "navigate", "target": "bedroom", "requires_confirmation": false, "reply": "OK"}'
)
```

三层稳定性保障：

1. 请求体带 temperature=0.0 和 response_format={"type": "json_object"}，让输出尽量确定且强制 JSON；
2. _strip_json_fence 剥掉模型常见的 ``json 围栏，再截取第一个 { 到最后一个 }；
3. parse_task_plan_payload 逐字段校验——**LLM 输出不可全信**：

```
# aliyun_clients.py -> parse_task_plan_payload 片段
intent = str(data.get("intent", "unknown")).strip().lower() or "unknown"
allowed = {"stop", "cancel", "confirm", "patrol", "follow",
           "mapping", "navigate", "move", "unknown"}
if intent not in allowed:
    intent = "unknown"          # LLM"发明"的新意图一律降级，防止下游崩溃
return AliyunTaskPlan(
    intent=intent,
    target=str(data.get("target", "") or ""),
    requires_confirmation=bool(data.get("requires_confirmation", intent == "unknown")),
    reply=str(data.get("reply", "") or fallback_text),  # 保证 TTS 总有话可播
)
```

口语如「帮我去卧室看看」规则可能吃力，LLM 更容易得到 navigate + 卧室。

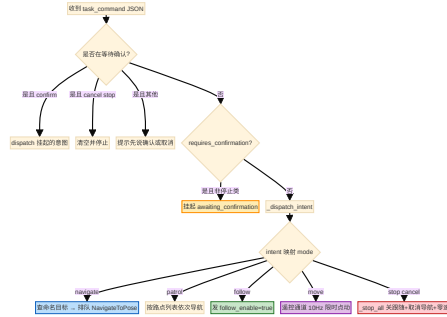
再次强调： LLM 输出的是计划，不是轮速。没有 mission_executor，JSON 发了车也不会动。

3.4 第三步：TaskCommand 如何变成动作

文件： mission_executor.py

订阅： /mecamind/task_command

执行入口状态机



确认闭环的真实代码 (`_task_cb` 核心分支):

```
# mission_executor.py -> _task_cb 片段
# 确认闭环：挂起中优先处理 confirm / cancel / stop。
if self._pending_confirm is not None or self._detail == "awaiting_confirmation":
    if intent == "confirm":
        pending = dict(self._pending_confirm or {})
        self._pending_confirm = None
        self._dispatch_intent(pending_intent, pending_target)
        return
    if intent in {"cancel", "stop"}:
        self._pending_confirm = None
        self._stop_all("stopped" if intent == "stop" else "canceled")
        return
# 挂起期间收到其他指令：提示先确认，不覆盖 pending。
self._publish_reply("请先说确认或取消。")
return

# 需要确认的指令先挂起；但 stop/cancel 例外——停止永远无条件执行。
if requires_confirmation and intent not in {"stop", "cancel", "confirm"}:
    self._pending_confirm = {"intent": intent, "target": target, "reply": reply}
    self._set_state("idle", "awaiting_confirmation", target)
    return
```

intent → 真实动作对照表

intent	next_mission_mode	具体动作	关键函数
navigate	navigate	中英文目标别名查表 → 排队发 NavigateToPose	_enter_navigate → _send_pending
patrol	patrol	读巡航路点 YAML，逐个导航	_enter_patrol → _goal_finished 推进
follow	follow	取消导航，/mecamind/follow_enable = true	_enter_follow
move	move	向 /cmd_vel_teleop 以 10Hz 限时连发速度	_enter_move → _move_tick
stop / cancel	idle	关跟随、cancel goal、双通道发零 Twist	_stop_all
confirm	—	执行挂起任务	_task_cb 确认分支
unknown 且需确认	—	只挂起，不派动作	_pending_confirm
mapping	idle	当前执行器不建图，等同不支持	回到 idle

范例 A：导航——命名目标查表与两阶段发送

命名目标是真实的地图坐标，配置在 config/mecamind_named_goals.yaml：

```
goals:
  hall_entry:  {frame_id: map, x: -3.6,  y: -2.7, yaw: 0.0}
  living_room: {frame_id: map, x: 0.0,   y: 0.6,  yaw: 0.0}
  kitchen:     {frame_id: map, x: -0.75, y: -1.8, yaw: 0.0}
  bedroom:     {frame_id: map, x: 3.75,  y: 0.9,  yaw: 0.0}
```

中文口语靠 resolve_named_goal 里的别名表落到英文键（注意「大厅」「门口」指向同一个点）：

```
# mission_executor.py -> resolve_named_goal
key = normalize_goal_key(target)      # 去空格、转小写、空格连字符统一成下划线
if key in goals:
    return goals[key]
aliases = {
    "客厅": "living_room",
    "卧室": "bedroom",
    "厨房": "kitchen",
    "大厅": "hall_entry",
    "门口": "hall_entry",
}
mapped = aliases.get(target.strip()) or aliases.get(key)
if mapped and mapped in goals:
```

```
    return goals[mapped]
return None                                # 查不到 -> detail 记 unknown_goal:原文
```

两阶段发送：收到 navigate 时不立刻调 Action，而是写入 _pending_send 排队，由 0.5s 定时器 _tick 在 Nav2 action server 就绪后再发。这样 Nav2 还在启动时指令不会丢，状态显示 waiting_for_nav2：

```
# mission_executor.py -> _tick 片段
if not self.navigate_client.server_is_ready():
    self._detail = "waiting_for_nav2"
    self._publish_state()
    return
self._send_pending()
```

真正发送时才生成时间戳（排队久了用旧时间戳会导致 TF 查询失败），并用**序列号**给回调"盖章"：

```
# mission_executor.py -> _send_pending 片段
stamp = self.get_clock().now().to_msg()
pose = pose_stamped_from_dict(pending["pose"], stamp)
goal = NavigateToPose.Goal()
goal.pose = pose
self._goal_serial += 1                # 每次发送都换新序列号
serial = self._goal_serial
send_future = self.navigate_client.send_goal_async(goal)
# lambda 默认参数在定义时刻捕获当前值，避免回调执行时读到已变化的变量
send_future.add_done_callback(
    lambda future, name=pending["name"], kind=pending["kind"], goal_serial=serial: (
        self._goal_response(future, name, kind, goal_serial)
    )
)
```

回调回来先验章，序列号对不上说明是**被取消的旧 goal** 的残留回调，直接丢弃：

```
# mission_executor.py -> _goal_finished 片段
def _goal_finished(self, future, name, kind, serial):
    if serial != self._goal_serial:
        return                # 旧 goal 的结果，忽略
    ...
    if status != GoalStatus.STATUS_SUCCEEDED:
        self._set_state("idle", f"failed:{name}")
        return
    if kind == "navigate":
        self._set_state("idle", f"reached:{name}", name)
```

发一条 去卧室 后 ros2 topic echo /mecamind/mission_state 会依次看到 (detail 里的名字保留中文原文)：

```
{"mode": "navigate", "last_intent": "navigate", "target": "卧室", "detail": "queued:卧室"}
{"mode": "navigate", "last_intent": "navigate", "target": "卧室", "detail": "sending:卧室"}
```

```
{"mode": "navigate", "last_intent": "navigate", "target": "卧室", "detail": "active:卧室"}
{"mode": "idle", "last_intent": "navigate", "target": "卧室", "detail": "reached:卧室"}
```

范例 B：方向点动——语音摇杆

这是本节新增的意图。说「前进」「左转」时，机器人以固定速度朝该方向动一小段时间，然后自动停——像按了一下摇杆：

```
# mission_executor.py -> _enter_move 片段
velocity = move_velocity_for_direction(direction, linear_speed, turn_speed)
# 点动与跟随/导航互斥：先清理干净其他模式的残留。
self._publish_follow(False)
self._cancel_nav()
twist = Twist()
twist.linear.x = float(velocity[0])
twist.angular.z = float(velocity[1])
duration_param = "turn_duration_sec" if direction in {"left", "right"} else "move_duration_sec"
duration = float(self.get_parameter(duration_param).value)
self._move_twist = twist
self._move_until = self.get_clock().now().nanoseconds / 1e9 + duration
self._set_state("move", f"moving:{direction}", direction)
```

```
# mission_executor.py -> _move_tick (10Hz 定时器)
def _move_tick(self) -> None:
    """10Hz 点动心跳：时限内连发速度，到时发零速并回 idle。"""
    if self._move_until is None:
        return
    now = self.get_clock().now().nanoseconds / 1e9
    if now < self._move_until:
        self.teleop_pub.publish(self._move_twist)
        return
    self._move_until = None
    self.teleop_pub.publish(Twist())
    if self._mode == "move":
        self._set_state("idle", f"moved:{self._target}")
```

为什么走 `/cmd_vel_teleop`（遥控通道）而不是导航通道？因为第五课的仲裁器里**遥控优先级高于跟随/导航**，且新鲜度窗口 0.5s——必须 10Hz 持续发才能保住通道；停发 0.5s 后仲裁器自动让位，**无需显式释放**。点动期间临时“抢”过控制权，动完自动归还。

默认参数（launch 可覆盖）：

参数	默认	效果
move_linear_speed	0.15 m/s	前进/后退速度
move_turn_speed	0.6 rad/s	转向角速度
move_duration_sec	2.0 s	前进/后退时长，约 0.3m
turn_duration_sec	1.5 s	转向时长， $0.6 \times 1.5 \approx$ 转 52°

还有一个实战兜底：LLM 有时把「后退」解析成 `navigate + target='backward'`，`_dispatch_intent` 里发现导航目标名是方向词就转成点动，而不是去查导航目标表：

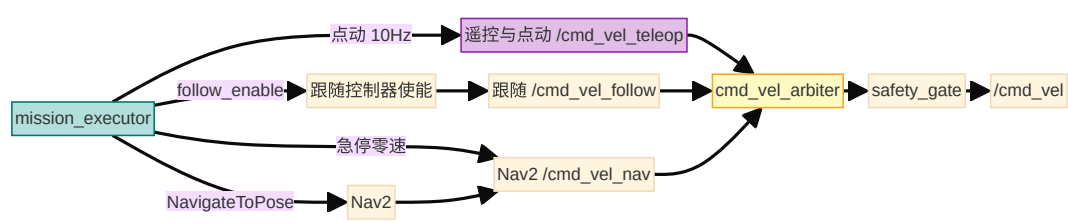
```
# mission_executor.py -> _dispatch_intent 片段
if mode == "navigate":
    direction = parse_move_direction(target)
    if direction:
        self._enter_move(direction)  # navigate target=backward -> 点动后退
        return
    self._enter_navigate(target)
```

范例 C：急停——为什么零速要发两个通道

```
# mission_executor.py -> _stop_all
def _stop_all(self, detail: str) -> None:
    self._pending_confirm = None
    self._publish_follow(False)      # 关跟随
    self._cancel_nav()               # 取消导航 goal 并作废旧回调
    self._move_until = None          # 终止点动
    self.cmd_pub.publish(Twist())     # 导航通道发零速：立即减速
    self.teleop_pub.publish(Twist()) # 遥控通道也发零速：若刚才在点动，立即刹停
    self._patrol_index = 0
    self._pending_send = None
    self._set_state("idle", detail)
```

发零 Twist 是为了让机器人**立即停下**，而不是等上游速度指令自然超时；两个通道都发，覆盖「正在导航」和「正在点动」两种情况。

与第五课速度链路的关系



mission_executor 通过**开关、Action 与遥控通道**控车，不与 PID 抢写同一话题；仲裁优先级 `estop > teleop > follow > nav` 保证点动能临时接管、急停永远生效。

3.5 第四步：回话闭环

task_scheduler 在发 task_command 的同时发 robot_reply → AliyunTtsNode 合成 wav → TtsPlaybackNode 用 ffmpeg / paplay / aplay 播放 → 状态回传 voice_listen (播放中暂停采麦，避免自己听自己)。

模糊指令的完整交互 (真实回复文案)：

1. 用户：随便做点什么
2. 规则 → unknown + requires_confirmation=true + reply 「请确认是否执行：随便做点什么？说确认或取消。」
3. executor 进入 awaiting_confirmation, **不派动作**
4. 用户：确认 → 执行挂起意图；或 取消 → 清空；说别的 → 回「请先说确认或取消。」

这正是 scripts/test_voice_core.sh 自动验收的路径。

4. 两种听模式与 PTT 兜底

4.1 continuous (默认, 产品形态)

voice_listen + aliyun_asr_stream: 能量触发 → 唤醒 → 最终文本发 voice_command。内置两个 WSL 自愈机制：

- 能量门限自动标定 (见 3.2 范例 1)；
- 采集看门狗：WSLg PulseAudio 采集流会偶发停滞 (尤其边播 TTS 边采麦)，capture_stall_sec (默认 5s) 内无数据就自动重启录音进程：

```
# voice_listen_node.py -> _capture_watchdog 片段
if stall_sec <= 0 or time.time() - self._last_chunk_at < stall_sec:
    return
self.get_logger().warning(
    f"Mic capture stalled >{stall_sec:.0f}s; restarting capture process"
)
```

4.2 ptt (按键说话)



```
ros2 launch mecamind_tools mecamind_aliyun_voice.launch.py listen_mode:=ptt
ros2 service call /mecamind/record_voice std_srvs/srv/Trigger
```

录音后端 auto: WSL 有 PULSE_SERVER 时优先 ffmpeg_pulse, 否则 arecord。实现见 select_microphone_backend。

5. 密钥、依赖与安全约定

本节云端 ASR / TTS / LLM 走**阿里云百炼 (DashScope)**。课堂实操前每人先开权限拿 Key；无 Key 仍可用 `llm_provider:=rule + 文本灌 voice_command` 验收规则链路。

5.1 阿里云开户与拿 Key (必做准备)

1. 用阿里云账号登录[百炼控制台 · API-KEY](#) (地域选北京即可)。
2. 创建 API-KEY，复制形如 `sk-...` 的字符串 (只显示一次，自行保存)。
3. 打开[免费额度](#)，确认语音识别 / 语音合成 / 通义千问等有可用额度；额度用尽会出现 401 / 余额类错误。
4. **不要开 thinking 模式** (深度思考 / 推理增强)。本课任务解析只要短 JSON，thinking 会拉长延迟、抬高消耗，且易把回复写成带推理过程的长文本，反而解析失败。控制台或模型页若出现「思考 / thinking」开关，保持关闭；本仓库默认模型为 `qwen-plus`，launch 未启用 thinking。

写入本机 (二选一，推荐文件)：

```
cp src/mecamind_tools/config/mecamind_aliyun.example.yaml \
  src/mecamind_tools/config/mecamind_aliyun.local.yaml
# 编辑 local: aliyun.api_key: sk-...
# 或: export DASHSCOPE_API_KEY=sk-...
```

- *.local.yaml 已 gitignore，**禁止提交真实密钥**，也禁止贴进群聊 / 截图发全 Key
- 云端 SDK: `pip install --user --break-system-packages -U dashscope`
- WSL 真麦 + 播报 (缺则装一次): `sudo apt install -y ffmpeg pulseaudio-utils`
- 课堂无网: `llm_provider:=rule`，文本灌 `voice_command`，仍可验收确认闭环

密钥查找顺序是真实代码 (`aliyun_clients.py`)：

```
# _require_api_key: 先查环境变量，查不到再读本地配置文件并写回环境变量
value = _env_value(env_name)          # 1. DASHSCOPE_API_KEY 环境变量
if not value:
    value = load_api_key_from_config(env_name) # 2. 候选 YAML 路径
    if value:
        os.environ[env_name] = value        # 写回，同进程后续调用不再读文件
```

候选 YAML 路径按优先级: `MECAMIND_ALIYUN_CONFIG` 显式指定 → 包源码旁 `config/` → 当前目录 `config/` → `~/.config/mecamind/`。找到文件但 Key 为空会**直接抛错**——「想配但配错了」应立刻提示，而不是让后续请求莫名失败。

默认模型参数 (Launch)：

能力	参数	默认
ASR	asr_model	paraformer-realtime-v2
LLM	llm_model / llm_provider	qwen-plus / aliyun
TTS	tts_model / tts_voice	cosyvoice-v3-flash / longanyang

6. 关键模块实现要点

6.1 voice_listen_node.py

函数 / 行为	作用
pcm_rms	16-bit PCM 能量
calibrated_energy_threshold	启动底噪自动标定门限
text_contains_wake_word / strip_wake_words	唤醒检测与剥词
_partial_cb	中间结果命中唤醒； 不 插播「我在」，以免掐断后半句
_result_cb	最终结果剥词后发 voice_command；纯唤醒词提示「请说具体指令」
_capture_watchdog	采集流停滞自动重启录音进程
TTS status 订阅	播放中暂停采集

6.2 aliyun_clients.py

符号	作用
AliyunLlmClient	标准库 urllib 调通义，解析任务 JSON
build_task_parser_messages	system 提示词约束 schema 与 intent 白名单
_strip_json_fence / parse_task_plan_payload	剥围栏 + 逐字段防御性校验
load_api_key_from_config / _require_api_key	密钥查找链
AliyunSpeechClient	文件 ASR / TTS
AliyunStreamingRecognition	流式 ASR WebSocket

6.3 aliyun_speech_nodes.py

节点	入口 executable
MicrophoneRecorderNode	mecamind_microphone_recorder
AliyunAsrFileNode	mecamind_aliyun_asr_file
AliyunAsrStreamNode	mecamind_aliyun_asr_stream
AliyunTtsNode	mecamind_aliyun_tts
TtsPlaybackNode	mecamind_tts_playback

另有 classify_voice_failure：把 ASR 空结果等异常映射成用户可读回复。

6.4 task_scheduler.py

符号	作用
TaskCommand	全系统统一任务结构
parse_task_command	零依赖规则 NLU，匹配顺序即优先级
parse_move_direction / MOVE_DIRECTION_WORDS	方向点动词表
task_command_from_aliyun_plan	LLM 结果隔离转换（换供应商只改这一处）
TaskSchedulerNode._parse_command	快指令走规则、云端优先、本地兜底

6.5 mission_executor.py

符号	作用
resolve_named_goal	英文键 + 中文别名
next_mission_mode	intent → 模式表，未知一律回 idle
_task_cb	确认闭环 + 分发
_dispatch_intent	intent → 模式入口，含 navigate 方向词兜底
_send_pending / _goal_response / _goal_finished	action 三环回调链
_goal_serial	序号号作废过期异步回调
_enter_move / _move_tick	遥控通道限时点动
_stop_all	双通道零速急停

6.6 project_state_machine.py

宏观项目模式（B00T / 建图 / 导航 / ESTOP 锁存），与语音 intent 状态机**分层**：前者管「整机阶段」，后者管「当前任务」。讲义不要求当堂改代码，但验收与简历可写「急停锁存优先级」。

7. 课堂实验 LAB

LAB-0 构建与单元测试

```
cd ~/mecamind_ros2
source /opt/ros/jazzy/setup.bash
colcon build --packages-select mecamind_tools --symlink-install
source install/setup.bash

python3 -m pytest src/mecamind_tools/test/test_mecamind.py \
-k "task_scheduler or wake or voice or aliyun_llm or move" -q
```

预期：相关用例通过（唤醒剥词、确认解析、方向点动、LLM payload、失败分类等）。

LAB-1 语音核心验收（必做，可不接仿真）

```
bash scripts/cleanup_ros2.sh
bash scripts/test_voice_core.sh
```

成功标志：MECAMIND_VOICE_CORE_OK

脚本做了什么：

1. 探测 API Key 可读（不打印全文）
2. 本地断言 确认/取消 解析
3. 以 listen_mode:=ptt、enable_microphone:=false、llm_provider:=rule、关 TTS 启动语音栈
4. 向 voice_command 发模糊句 → 检查 requires_confirmation 与回复含「确认」
5. 再发「确认」→ 检查 intent == confirm

LAB-2 文本路径手调调度器

```
ros2 launch mecamind_tools mecamind_aliyun_voice.launch.py \
listen_mode:=ptt enable_microphone:=false llm_provider:=rule
```

另开终端，把 2.3 节对照表逐条打一遍：

```
ros2 topic echo /mecamind/task_command
ros2 topic echo /mecamind/robot_reply

ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '去卧室'}"
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '前进'}"
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '左转'}"
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '开始跟随'}"
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '停止'}"
```

```
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '随便做点什么'}"
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '确认'}"
```

观察 JSON 的 intent/target/reply 与对照表一致。

可选：ros2 param set /mecamind_task_scheduler provider aliyun（运行中切，立刻生效），再说「帮我去卧室看看」，对比规则与 LLM 的 target/reply；断网重试观察「已改用本地理解」降级前缀。

LAB-3 连续听 + TTS（有麦）

```
ros2 launch mecamind_tools mecamind_aliyun_voice.launch.py
# 说：小度小度去卧室
# 听：TTS 回复；echo task_command
```

无唤醒调试可临时改 wake_words:= 为空（任何语音都算唤醒），或继续用文本 pub。

LAB-4 真正让车动起来（选做 / 综合演示）

需要：已有地图、Nav2 可导航；跟随链路按第五课。

推荐顺序：

1. 按第三节启动导航（或项目 bringup 导航入口）
2. 启动交互层（含 scheduler + executor）：

```
ros2 launch mecamind_tools mecamind_interaction.launch.py \
  task_provider:=rule enable_fake_detections:=true
```

3. 再开语音，或直接文本：

```
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '去卧室'}"
ros2 topic echo /mecamind/mission_state
```

预期 mission_state 依次经过 queued:卧室 → sending:卧室 → active:卧室 → reached:卧室（见 3.4 范例 A）；RViz 出现导航目标。

方向点动（不依赖 Nav2，起了 executor + 仲裁链路即可）：

```
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '前进'}"
# mission_state: mode=move detail=moving:forward, 2 秒后 detail=moved:forward
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '右转'}"
# 1.5 秒点动，约转 52 度
```

跟随与急停：

```
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '开始跟随'}"
# mission_state: mode=follow detail=follow_enabled
ros2 topic pub --once /mecamind/voice_command std_msgs/msg/String "{data: '停止'}"
# mission_state: mode=idle detail=stopped, 双通道零速立即刹停
```

历史综合入口 mecamind_demo_ui.launch.py 可一并拉导航+交互；开麦时 enable_microphone:=true，且该入口里语音 launch 关闭了内置 scheduler（避免与 interaction 双重调度），以 interaction 侧为准。

LAB-5 跟随仿真 + 语音控车 + 摄像头监视（课堂主演示）

场景：第五课跟随 demo 已能跑；本节叠语音：唤醒后说「跟随我 / 前进 / 后退 / 左转 / 右转 / 停止」，并用图像窗看检测框。

前提：§ 5.1 已配好 Key；本机有麦；每个新终端都要 source install/setup.bash（系统 ROS 若已在 ~/.bashrc 里 source 过，可只补工作区那一句）。

先清理干净再起（避免旧 Gazebo / 脏 Fast DDS 共享内存导致「有 JSON 车不动」）：

终端 1 — 清理 + 启动仿真跟随

```
cd ~/mecamind_ros2
pkill -9 -f "gz sim"; pkill -9 -f "ros2 launch"; pkill -9 -f mecamind_; sleep 2
rm -f /dev/shm/fastrtps_* /dev/shm/fast_datasharing_* /dev/shm/sem.fastrtps_*
source /opt/ros/jazzy/setup.bash && source install/setup.bash
ros2 launch mecamind_bringup follow.launch.py \
  backend:=gazebo source:=camera infer_backend:=hsv moving_target:=true
```

终端 2 — 任务执行器（纯语音 launch 默认不起它；没有它则只会说话、车不动）

```
source /opt/ros/jazzy/setup.bash
source ~/mecamind_ros2/install/setup.bash
SHARE=$(ros2 pkg prefix mecamind_tools)/share/mecamind_tools
ros2 run mecamind_tools mecamind_mission_executor --ros-args \
  -p named_goals_file:=$SHARE/config/mecamind_named_goals.yaml \
  -p waypoints_file:=$SHARE/config/mecamind_patrol_waypoints.yaml
```

若日志出现 Failed init_port fastrtps_port...，可先再清一遍 /dev/shm/fastrtps_*，或本终端强制 UDP：

```
FASTDDS_BUILTIN_TRANSPORTS=UDPv4 ros2 run mecamind_tools mecamind_mission_executor --ros-args \
  -p named_goals_file:=$SHARE/config/mecamind_named_goals.yaml \
  -p waypoints_file:=$SHARE/config/mecamind_patrol_waypoints.yaml
```

终端 3 — 语音栈（日志留在此终端，方便和老师对表）

```
cd ~/mecamind_ros2
source /opt/ros/jazzy/setup.bash && source install/setup.bash
ros2 launch mecamind_tools mecamind_aliyun_voice.launch.py
```

试说（一口气）：小度小度，停止 / 小度小度，跟随我 / 小度小度，前进 / 小度小度，后退 / 小度小度，左转 / 小度小度，右转。

期望日志链：Wake word heard → Voice command: '...' → intent=... → TTS synth → Playing TTS audio；执行器侧 mission_state 同步变化。

终端 4 — 摄像头监视（WSLg 用 :0；若 ~/.bashrc 里还残留旧 DISPLAY=某 Windows IP:0，先注释掉再开新终端）

```
source /opt/ros/jazzy/setup.bash && source ~/mecamind_ros2/install/setup.bash
export DISPLAY=:0
ros2 run rqt_image_view rqt_image_view /mecamind/detection_image
```

窗口若黑屏，用顶部下拉框手动选 /mecamind/detection_image。

8. 验收清单与故障排查

8.1 本节验收

项	命令 / 现象	通过标准
单元	pytest 语音相关	绿
核心 smoke	test_voice_core.sh	MECAMIND_VOICE_CORE_OK
文本→任务	pub 去卧室	task_command 含 navigate + 卧室类 target
点动	pub 前进 / 左转	intent=move + 正确方向；接执行器则车动 2s 后停
确认闭环	模糊句 → 确认	先挂起再 confirm
云端可选	provider=aliyun	口语可解析；失败有「已改用本地理解」降级
综合可选	接 Nav2	mission_state 反映 reached；停止立刻 idle
主演示	LAB-5 四终端	唤醒后点动/跟随/停止生效；检测图可见

8.2 常见故障

现象	可能原因	处理
smoke 报缺 Key	未配 local / 环境变量	按 § 5.1 开百炼 Key 后复制 example 或 export
401 / 无额度	免费额度用尽或 Key 错	查 免费额度 ；换 Key
LLM 很慢 / 回复不像 JSON	开了 thinking	控制台关闭思考模式；确认 launch 未改 thinking 相关参数
有 JSON 车不动	只起了语音栈；或 DDS SHM 坏了	起 mission_executor；清 /dev/shm/fastrtps_*；必要时 FASTDDS_BUILTIN_TRANSPORTS=UDIPv4
Package not found	新终端未 source 工作区	source ~/mecamind_ros2/install/setup.bash
rqt_image_view 连不上 display	旧 DISPLAY 指向 Windows IP	export DISPLAY=:0；修 ~/.bashrc
unknown_goal	目标名不在别名表	用客厅/卧室/厨房/大厅或英文键
点动没动静	仲裁器没起 / teleop 话题不通	查 /cmd_vel_teleop 有无 10Hz 输出
唤醒不灵	能量门限过高 / 无麦	看标定日志；调低 energy_threshold；改用文本
TTS 时麦卡死	WSLg Pulse 停滞	看门狗自动重启；或 PTT
「后退」变导航	LLM 解析成 navigate backward	_dispatch_intent 已兜底转点动，属预期行为
跟随时还在导航	模式未互斥清理失败	先说停止；查 mission_state
LLM 慢或失败	网络 / Key	自动降级 rule；确认/停止/点动短指令本就走规则

清理残留：

```
bash scripts/cleanup_ros2.sh
# 若仍有「有 JSON 车不动」，再清一次 DDS 共享内存：
rm -f /dev/shm/fastrtps_* /dev/shm/fast_datasharing_* /dev/shm/sem.fastrtps_*
```

9. 项目收尾：演示脚本与简历表述

9.1 两分钟演示脚本建议

- 1. 课前：§ 5.1 开好百炼 Key；按 LAB-5 起齐四终端
- 2. 文本路径（可选热身）：模糊指令 → TTS/回复「请确认」→ 「确认」→ 状态变化
- 3. 点动：小度小度，前进 / 左转 各来一下，展示语音摇杆
- 4. 小度小度，跟随我 → 红柱移动、检测窗有框 → 小度小度，停止

5. 有地图时再加：去厨房 → RViz 路径 → 到达 (LAB-4)

9.2 简历可写要点（对齐真实代码）

- 基于 ROS 2 的语音任务链路：流式 ASR、规则/LLM 双 Provider、结构化 TaskCommand
- LLM 失败自动降级；确认/急停/点动短指令强制本地解析，保证低延迟零歧义
- mission_executor 将 intent 映射到 Nav2 Action、跟随使能、遥控通道点动与双通道急停，与速度仲裁解耦
- action 异步回调链用序列号作废过期回调，两阶段发送容忍 Nav2 晚启动
- 无麦文本路径与 test_voice_core.sh 保证课堂/CI 可验收

10. 源码阅读顺序（课后）

1. task_scheduler.py: parse_task_command → _parse_command
2. aliyun_clients.py: build_task_parser_messages → parse_task → parse_task_plan_payload
3. mission_executor.py: _task_cb → _dispatch_intent → _enter_navigate / _enter_move
4. voice_listen_node.py: 唤醒状态机与 _result_cb
5. mecamind_aliyun_voice.launch.py: 节点如何拼成一条链

附录 A 话题速查

名称	类型	说明
/mecamind/audio_pcm	String	base64 PCM 块 (含 seq / rms / phase)
/mecamind/audio_pcm_end	String	结束流式会话
/mecamind/asr_partial	String	中间识别
/mecamind/asr_result	String	最终识别
/mecamind/listen_status	String JSON	听麦状态 idle/wake/capturing
/mecamind/voice_command	String	自然语言指令
/mecamind/task_command	String JSON	结构化任务
/mecamind/robot_reply	String	TTS 文案
/mecamind/tts_audio_file	String	wav 路径
/mecamind/tts_status	String	播放状态
/mecamind/mission_state	String JSON	执行状态心跳 0.5s
/mecamind/follow_enable	Bool	跟随开关
/cmd_vel_teleop	Twist	点动 10Hz 输出通道
/controller/cmd_vel_nav	Twist	急停零速通道 (executor 默认 cmd_vel_topic)
/mecamind/record_voice	Trigger 服务	PTT 录音

附录 B 与规划文档的对应

PROJECT_PLAN.md 10.3.6: 麦克风 / ASR / LLM / TTS、导航·巡逻·跟随·停止、云端失败兜底、一键验收。

本讲义以仓库已实现的 `mecamind_aliyun_voice + task_scheduler + mission_executor + test_voice_core.sh` 为准；方向点动（move intent）为课程迭代新增能力，已纳入 intent 白名单与验收清单。§ 5.1 / LAB-5 补齐课堂实操缺口：百炼开 Key、免费额度、勿开 thinking、跟随+语音+摄像头四终端联调。全仓 test_cp6 若后续补齐，验收命令以 README 更新为准。